
From Pull Requests to Sandboxed Restoration: A Logbook of Building an LLM Hardware-Agent Benchmark

David Andrews*, Saketh Reddy*
Georgia Institute of Technology
{dandrews47, sreddy347}@gatech.edu

Abstract

This logbook documents two and a half months of work building an LLM-agent benchmark for hardware description languages, ending with the RTL-UNIVERSE benchmark we report in our companion paper. The arc covers four prior repositories — a GitHub-PR data pipeline (SWE-Universe-Verilog), an early eval harness (RTL Bench), a port of NVIDIA’s CVDP problem suite into our infrastructure (CVDP-Harbor), and a spec-to-GDSII benchmark we built and iterated weekly through three pivots (Spec2GDS Bench) — before the move to project-scale *restoration* of open-source HDL projects (RTL-UNIVERSE). We write the logbook as a class progress report: what we tried, what worked, and where we pivoted, with concrete references to commits, weekly reports, and run results that are still in the project’s `other_repos/` directory.

1 Goal

The high-level question we set out on was: *can current LLM agents do real hardware-engineering work, and how do we measure that?* The literature on hardware LLMs in early 2026 was almost entirely module-level Verilog generation [Liu et al., 2023, Lu et al., 2024, Thakur et al., 2023], with leading models reporting 90%+ pass rates on benchmarks that ask for “a 4-bit counter from a description.” That left two open questions: how would those agents fare on the rest of a real chip-design workflow (build systems, integration, testbenches, physical design), and is there a benchmark that we trust to grade them on it?

Over two and a half months we built five repositories, in roughly chronological order: SWE-UNIVERSE-VERILOG, RTL BENCH, CVDP-HARBOR, SPEC2GDS BENCH, and RTL-UNIVERSE. Each one taught us something that we then either folded into the next or used as the reason to start from scratch. The rest of this document is that arc.

2 March 2–6: SWE-Universe-Verilog (data pipeline)

What we tried. SWE-bench [Jimenez et al., 2024] grades agents on real GitHub bugs in real Python projects. The first thing we tried was the same recipe but for HDL. The SWE-UNIVERSE-VERILOG pipeline crawls GitHub for repositories whose primary language is Verilog or SystemVerilog (or that match HDL topics: `verilog`, `fpga`, `rtl`, `hdl`, `asic`), collects all merged pull requests, and applies a multi-stage filter inspired by SWE-Universe’s approach [Anthropic, 2026].

Pipeline yield.

*Equal contribution. Authors listed in alphabetical order.

Stage	Count
Repos discovered (Verilog/SV on GitHub)	3,296
PRs collected (merged)	173,106
After heuristic filter (size, bot, merge)	27,446
After testbench detection	5,282
After patch separation (test + fix files)	3,070
Quality filter (relaxed)	1,241
Quality filter (strict, with issue linkage)	401

What we learned. Even after aggressive filtering, the yield from the GitHub HDL ecosystem is roughly an order of magnitude lower than the Python equivalent. 401 strict-filter PRs across 3,296 repositories is enough to start a benchmark, but each one had to be curated by hand to be sure it isolated a single change with a real before-and-after test. We built RTL_{BENCH} (§3) on top of this data and quickly discovered that PR-based hardware bugs do not, in practice, isolate cleanly: most are part of a larger refactor, depend on FPGA-vendor toolchains, or test only a manually-asserted simulation trace.

This phase informed two decisions in RTL-UNIVERSE: we chose to abandon PR-replay as the framing, and we chose *restoration* (delete-and-rebuild) instead, because it gives the agent a real test suite (the upstream’s own CI) without requiring us to curate a clean before-and-after diff.

3 March 2–3: RTL_{Bench} (first agent runs)

What we tried. Using the SWE-Universe-Verilog filtered set, plus a few additional integration projects (corundum networking, CVA6 RISC-V core), we built a quick eval harness called RTL-_{BENCH}: clone the upstream repo at the parent commit of a chosen PR, set up the toolchain in a container, and ask Claude to make the change. The early commits were exploratory — “init,” “snap,” “first claude run,” “claude plan expand not run yet,” “ran full eval” (commits 45e61b9–38a68a1).

What we learned. The first runs worked just well enough to show two failure modes that ended up shaping everything that followed: (1) **Toolchain fragility**: Verilog projects often pin specific Verilator/Yosys versions, and the agent would happily install a different one and write code that compiled in the wrong version. We ended up writing per-task Dockerfiles, which is what the next two projects (CVDP-Harbor and Spec2GDSBench) inherit. (2) **Test-suite ambiguity**: many of the PRs in our filtered set have testbenches that “run” on master but actually only pass when accompanied by a manually-checked waveform. The agent does not know this. We needed verifiers whose pass criterion is mechanically checkable. This is part of why RTL-UNIVERSE restricts itself to projects with binary-pass CI.

4 March 7–13: CVDP-Harbor

What we tried. At this point NVIDIA released the Comprehensive Verilog Design Problems (CVDP) benchmark [NVIDIA Research, 2024], a 783-problem suite of mostly module-level Verilog tasks with both non-agentic and agentic variants. Rather than re-invent the problems, we ported CVDP into *Harbor*, our internal evaluation harness used by the rest of the projects. The conversion tool is at `cvdp_benchmark/` commit `c002eb5`, with three more rounds of fixes and reruns over the next week.

Result. We ran three Claude variants on the converted suite. Pass rates clustered above 90% (`cvdp_benchmark` commit `891039d`), consistent with what NVIDIA themselves reported. This was when we became confident that *module-level Verilog generation is saturated for current frontier models* and that the open question is at the project scale. At this point we stopped working on CVDP and started a benchmark we hoped would not saturate.

5 March 23–April 12: Spec2GDSBench (three weekly pivots)

Goal. Given a behavioural specification and a test suite inside a containerised environment with the full open-source ASIC toolchain (LibreLane + SkyWater Sky130A PDK), ask the agent to au-

tonomously produce a DRC-clean, LVS-verified GDSII file. The grading funnel is: functional simulation passes → Yosys synthesis → LibreLane place-and-route → KLayout/Magic DRC → Netgen LVS → GDSII export. Each stage gates the next.

Week 1 (Mar 24–28): eight isolated tasks. The first version had eight tasks: `gcd_unit`, `uart_tx`, `aes128_encrypt`, `femtorv32_rv32i`, and four AI accelerator primitives (`conv1d_engine`, `maxpool2d`, `requantize_pipeline`, `matmul_4x4_int8`). 64 runs of Claude Code Sonnet at \$5/run got mean reward 0.964 in normal mode, 0.829 in blind mode (no test suite visible). All 32 normal-mode runs passed all functional tests; the only score loss was timing closure during physical design [Andrews and Reddy, 2026a]. The benchmark was near-saturated. We pivoted.

Week 2 (Mar 31–Apr 6): single SoC, Linux boot. We replaced the eight tasks with one task: design an RV64GC RISC-V SoC capable of booting Linux. The agent receives only the external interface (AXI4 + UART) and a software contract (108 ISA compliance tests, 10 bare-metal programs, OpenSBI/BusyBox boot via QEMU’s virt machine convention). Claude Sonnet 4.6 across five runs scored a best of 0.616 (85/108 ISA, 10/10 bare-metal, OpenSBI boot). A 15-hour run reached 99/108 ISA and showed partial Linux boot during debugging [Andrews and Reddy, 2026b]. This was a much harder benchmark, but a single task gave poor signal: an agent either built a working SoC or scored near zero. We pivoted again.

Week 3 (Apr 7–12): AI accelerator integration ladder. We then designed three new tasks at graduated difficulty, all with AXI4 interfaces: `systolic_array` (8×8 INT8 MAC array, AXI4-Lite MMIO + AXI4-Stream), `conv2d_engine` (3×3 convolution with line buffer, AXI4 master), `dma_engine` (memory-to-memory DMA, AXI4-Lite + AXI4 master). All three reference solutions pass the full ASIC flow through GDSII; only the DMA meets timing at 50 MHz [Andrews and Reddy, 2026c]. We were partway through getting agent runs on these when the next pivot happened.

What we learned across the three weeks. Specifications are hard. Every week spent on Spec2GDSBench, we spent some fraction of the time on whether the spec was over-constraining the agent (forcing a specific microarchitecture) or under-constraining it (letting the agent solve a different problem). Real industrial specs are often closer to “here is a JIRA epic with 30 linked tickets” than “here are the AXI4 signal names.” We could not write a clean spec for a realistic design.

Build systems matter as much as RTL. The agent’s score on Spec2GDSBench was dominated by physical design, not RTL design — timing closure, place-and-route choices, LibreLane configuration. We wanted RTL-design ability to be the bottleneck.

Real projects already have all of this. The thing that finally moved us was realising that lowRISC’s Ibex, NVIDIA’s NVDLA, OpenPiton, and the rest already have testbenches, build systems, CI configurations, and reference implementations carefully tuned by their maintainers. We do not need to write a spec for a UART — lowRISC has a real one and grades it with their own `simple_system`. We just need to delete files.

6 April 23 – May 8: RTL-Universe

What it is. RTL-UNIVERSE is the result of all the prior decisions: real upstream projects, real upstream CI, no spec to write, restoration as the framing. The task list is in our companion paper. The harness builds on the Harbor runner that started in CVDP-Harbor (§4); the Docker patterns follow Spec2GDSBench’s three-layer image; the verifier conventions are upstream’s CI.

The first three weeks. Most of late April and early May was task engineering — we wrote a `harbor-task-gen` skill (commits 1ad3b4d, 82ce12f, 1df25f5) that converts an arbitrary upstream HDL repo into a Harbor task with correct file-strip manifest, environment Dockerfile, and verifier script. The skill went through five iterations to handle five different upstream build systems (FuseSoC, Bazel, Make+Verilator, Icarus, cocotb). We added Ibex, Caliptra, PULP common_cells, secworks-aes, VeeR EL2, CVA6, Coral NPU, NVDLA, and OpenPiton. Roughly half of the tasks we tried did not make the final eleven — some had upstream build systems that the agent couldn’t

drive at all (Caliptra), some had test suites with too much state for the verifier to be deterministic (CVA6 in some configurations).

The runs. The headline run on the eleven tasks took roughly four days of wall-clock and seven agent×harness combinations: Opus 4.7 and Sonnet 4.6 via Claude Code, GPT-5.5 (effort xhigh) via Codex CLI, and DeepSeek V4-Pro, GLM-5.1, Kimi K2.6, Qwen 3.6-27B via Open-Code/OpenRouter. The harness needed several mechanisms we had not anticipated when we started: per-account concurrency caps because Claude Code rate-limits per subscription, a disk supervisor because a Sonnet run leaked 812 GB of Verilator build cache through a bind-mounted credentials directory and would have crashed the lab server, and a stuck-job killer because the agents sometimes write at human pace for hours and then freeze.

The result, briefly. The headline numbers are 6/11 perfect for GPT-5.5 and 5/11 for Opus 4.7, with the smaller open-weight models clustering at 0–2 perfects each. Reading the transcripts we found that most of the perfect scores came either from sandbox bypasses (CDN mirrors, IP-literal DNS bypass, vendor-side fetch tools) or from two upstream verifier flaws (an exit-code-only test in lbex and an `assert True` scoreboard in VeeR-EL2). The strong-verifier pass rate is much lower: 1–2/11 for the best agents, 0/11 for the weakest. The full discussion is in the companion paper.

7 What we would do differently

Three lessons we would carry into the next iteration:

Sandbox first. We built the harness and added the DNS sinkhole later. By the time we audited the transcripts, most of the perfect scores had already been bypassed. A default-deny egress firewall with explicit allowlists, plus an audit pass that flags external network egress in real time, would have caught everything in this paper before the run completed.

Verifier hardening should be part of task acceptance. For every task we adopt from an upstream project, “does the agent score 1.000 with a stub” should be a check we run before greenlighting the task. Two of the three flagged tasks were exploitable in 17 minutes of agent time; we could have caught both during task setup.

Restoration is the right framing for now. Of the four prior approaches (PR-replay, CVDP module synthesis, Spec2GDSBench, restoration), restoration is the only one that simultaneously (a) gives the agent a real test suite, (b) avoids the specification problem entirely, (c) scales to project-scale work without us having to write build infrastructure, and (d) produces a benchmark whose top-line we trust. RTL-UNIVERSE as it stands is not the right benchmark forever, but the restoration framing has more headroom than what we had before.

Acknowledgements. We thank the lowRISC, chipsalliance, NVIDIA, Princeton, PULP, Google Coral, and secworks teams for the upstream projects this benchmark is built on. Repository contributions over this arc are split roughly evenly between the two authors, with several small contributions appearing under our other GitHub handles (Brojojo and spyre respectively, plus the SpiritSeal handle that was used during the early SWE-Universe-Verilog and RTL Bench work).

References

- David Andrews and Saketh Reddy. Spec2GDSBench: Implementation and baseline results for end-to-end chip design evaluation. Technical report, Internal weekly research report, March 2026a.
- David Andrews and Saketh Reddy. From individual tasks to full SoC: Redesigning spec2GDSBench around linux boot. Technical report, Internal weekly research report, April 2026b.
- David Andrews and Saketh Reddy. Pivoting to AI accelerator benchmarks: An integration ladder for spec2GDSBench. Technical report, Internal weekly research report, April 2026c.
- Anthropic. SWE-Universe: A multi-stage filtering methodology for software-engineering benchmarks. arXiv:2602.02361, 2026.

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation. *arXiv preprint arXiv:2309.07544*, 2023.
- Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RTLLM: An open-source benchmark for design RTL generation with large language models. *IEEE Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024.
- NVIDIA Research. CVDP: A comprehensive verilog design problems benchmark. <https://github.com/NVlabs/CVDP-benchmark>, 2024.
- Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. VeriGen: A large language model for Verilog code generation. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 29(3), 2023.